Web Application Fuzzer

This is for Prof. Basham's and Prof. Lukyanov's classes. Prof. Meneely's section use the beta Fuzzer instructions (/~swen-331/projects/fuzzer-beta) and beta web app instructions (/~swen-331/activities/webapps-beta).

Fuzzer Project Overview

Overview

This project is for individuals.

One of the most helpful tools that a security-minded software developer can have is a fuzz-testing tool, or a **fuzzer**. A fuzzer is a typ program by scanning its attack surface.

The best fuzzers are *highly* customizable, so generalized fuzzers are often quite complex to configure and use, and can become out· a fuzzer that can be customized to a specific web application rapidly.

Programming Language

You have two choices of programming language: Ruby or Python. You may choose the specific packages to use - research them and For Ruby, successful projects have used Ruby with Mechanize (https://rubygems.org/gems/mechanize/), or Python with the Reque Mechanical Soup (https://github.com/MechanicalSoup/MechanicalSoup). If you are using Python, past students strongly recomme packages (i.e. use “python3” and “pip3” to run since Python 2 is long dead (https://www.python.org/doc/sunset-python-2/)).

Think of the above libraries like a GUI-less browser - it can simulate everything that a browser does, but programmatically. It does I

- Handle the HTTP protocol, so you don’t have to worry about networking details
- Parse HTML, so you can find inputs via relatively simple XPATH queries or something similar
- Simulate user actions, such as clicking links and submitting forms

Your code will be tested against a modified version of the DVWA we provided in the web application activity. We recommend you us enough to work on any website.

NOTE: The root URL for the CI image is slightly different from what you most likely install on your local machine. Specifically, the rc image (not `http://localhost/dvwa`). This different is intentional, and your fuzzer should be able to handle that flexibility since the

Starter Examples

This example demonstrates a quick script for getting the links from our course web page using Ruby and the Mechanize gem:

```ruby
require 'mechanize' # you'll need Mechanize installed (gem install mechanize)

# This is some example code that just grabs all the links on a page
# More docs can be found here: http://docs.seattlerb.org/mechanize

agent = Mechanize.new
url   = 'http://www.se.rit.edu/~swen-331'
puts "Visiting #{url}"
agent.get(url).links.each do |link|
  puts "Link: #{link.uri}"
end

# Are you getting this error?
# `<class:Persistent>': uninitialized constant Process::RLIMIT_NOFILE (NameError)
# On Windows there's a known issue at the moment (https://github.com/sparklemotion/mechanize/issues/529)
# You can read about the workaround there
```

Here is some Python that uses MechanicalSoup:

```
import mechanicalsoup

# Connect to our course website
browser = mechanicalsoup.StatefulBrowser(user_agent='MechanicalSoup')
browser.open("http://www.se.rit.edu/~swen-331")

# Find all links using the CSS selector
for link in browser.page.select('a'):
    print(link.text)
```

Command-Line interaction

Your fuzzer must run from the command line. We recommend using a command line parsing library, such as Python's argparse (http (https://docs.python.org/3/library/optparse.html), click (https://click.palletsprojects.com/en/stable/), or Ruby's option parser (h 2.6.3/libdoc/optparse/rdoc/OptionParser.html) (or any library you wish to add).

Depending on your language, your exact command might vary (e.g. `python fuzz.py` or `ruby fuzz.rb`), but the basic structure shc

```
fuzz [discover | test] url OPTIONS

COMMANDS:
  discover  Output a comprehensive, human-readable list of all discovered inputs to the system. Techniques include both crawling and gues
  test      Discover all inputs, then attempt a list of exploit vectors on those inputs. Report anomalies that could be vulnerabilities.

OPTIONS:
  Options can be given in any order.

  --custom-auth=string     Signal that the fuzzer should use hard-coded authentication for a specific application (e.g. dvwa).

  Discover options:
    --common-words=file    Newline-delimited file of common words to be used in page guessing. Required.
    --extensions=file      Newline-delimited file of path extensions, e.g. ".php". Optional. Defaults to ".php" and the empty string if n

  Test options:
    --common-words=file    Same option as in discover - see above.
    --extensions=file      Same option as in discover - see above.
    --vectors=file         Newline-delimited file of common exploits to vulnerabilities. Required.
    --sanitized-chars=file Newline-delimited file of characters that should be sanitized from inputs. Defaults to just < and >
    --sensitive=file       Newline-delimited file data that should never be leaked. It's assumed that this data is in the application's d
    --slow=500             Number of milliseconds considered when a response is considered "slow". Optional. Default is 500 milliseconds
```

Example invocations:

```
# Discover inputs, default extensions, no login
fuzz discover http://localhost:8080 --common-words=mywords.txt

# Discover inputs to DVWA using our hard-coded authentication, port 8080
fuzz discover http://localhost:8080 --custom-auth=dvwa --extensions=extensions.txt --common-words=mywords.txt

# Discover and Test DVWA, port 8000, default extensions: sanitized characters, extensions and slow threshold
fuzz test http://localhost:8000 --custom-auth=dvwa --common-words=words.txt --vectors=vectors.txt --sensitive=creditcards.txt
```

Example Output

Your output should be human readable. Think of it like a build report you might get in an email that you can review from time to tim vulnerabilities, and it should also be readable enough that you're trying to read through HTTP outputs and stacktraces.

An example of well-formatted output for the `discover` can be found here (example-output.txt). (You do not need to match this form here (example-output-test.txt). **Note** that DVWA and our requirements evolve over time, so these exact outputs may be slightly diff outputs, not oracles for expected results.

Test Environments

DVWA. Use the DVWA download from our web application activity as your main test environment. You can use the zip file from our 331/activities/webapps/).

fuzzer-tests. Additionally, we have created a set of simpler test cases at the `/fuzzer-tests` url in the DVWA zip file and on the Doc `/htdocs/fuzzer-tests`. Your fuzzer should be able to find the following:

...during `fuzz discover`:

- The main page `index.php` has two inputs, one called `calzone` (discoverable via form) and another called `message` (discoverab
- There is a page called `valid.php`
- There is a page called `timeout.php`
- There is a page called `admin.php` not linked from anywhere, it has an input called `company` (discoverable via form OR url parsi
- There is a page called `sensitive.php` linked only from `admin.php`
- There is NOT a page called "CioffisTheBest.html". That link is dead.
- Your fuzzer should NOT go in an infinite loop.
- Your fuzzer should NOT go `http://se.rit.edu`

...during `fuzz test`:

- `index.php`, input `calzone` lacks sanitization
- `index.php`, input `message` lacks sanitization
- `valid.php` has no inputs
- `timeout.php` has no inputs, but it has a long delay
- `admin.php`, input `company` lacks sanitization
- `sensitive.php` has no inputs, but leaks sensitive data of 123-45-6789
- `timeout.php` takes at least 2 seconds to load

These test cases are maintained over at this GitHub project (https://github.com/andymeneely/swen331fuzzerCI). Pull requests wel

Submission Instructions

You must use RIT's installation of GitLab for this project. By a pre-determined date (given by your instructor), please do the followin

- Go to https://git.gccis.rit.edu/ (https://git.gccis.rit.edu/) (NOT kgcoe-git and NOT gitlab.com!!)
- Use your RIT login to sign in ("LDAP Login").
- Create a new project - be sure to have the word "fuzzer" in the title. **Visibility is Private** (please do not share your code, even a
- Now add the instructor and CA to your project:
  - On the project page use the *Manage* > *Members* menu option.
  - The system will display the *Project members* page. Click on the *Invite members* button (upper-right).
  - In the *Invite members* dialog: (a) enter the instructor's name and select the appropriate user, (b) select *Reporter* in the *Role*
  - Follow the same instructions to add your section's Course Assistant.

You are required to push your code to this repository by the deadline. At each deadline, we will automatically pull the code and grac keep working on the same repository for the entire fuzzer project. Each time you finish a submission, create a Tag in the `master` br

Please include a file called `.gitlab-ci.yml` (note the dot at the beginning of the file name) in the root of your repository. Here is th Note that YML files don't like tabs as whitespace and are finicky about number of spaces for indentation.

```
image:
  name: andymeneely/swen331fuzzer # don't change this
  entrypoint: [""]  # don't change this
before_script:
  # don't change these either
  - chown -R mysql:mysql /var/lib/mysql /var/run/mysqld
  - echo '[+] Starting mysql...'
  - service mysql start
  - echo '[+] Starting apache'
  - service apache2 start
fuzzrunner:
  script:
    # here is where you can write your commands to run your fuzzer or any custom setup commands
    - echo "hello class"
    # need some example files for vectors and words? These are on the image
    - cat /words.txt
    - cat /vectors.txt
    - cat /badchars.txt
    # An example fuzzer command. Note the url is DIFFERENT than XAMPP example (no /dvwa).
    # Remove whatever you need to.
    - ruby fuzz.rb discover http://localhost/ --custom-auth=dvwa
    - ruby fuzz.rb discover http://127.0.0.1/fuzzer-tests
    - python3 fuzz.py discover http://localhost/ --custom-auth=dvwa
    - python3 fuzz.py discover http://127.0.0.1/fuzzer-tests
  stage: test
```

This is a continuous integration configuration file. Every commit you push to the repository, your fuzzer will be run against DVWA i find your build in the "Pipelines" page. *You are strongly encouraged to keep an eye on this output to make sure your code is working as*

Your application should be easy to use from a customer's perspective. Some notes about your submision include:

- Fuzzers will be run on our docker image fuzzer-tests and DVWA (https://hub.docker.com/r/andymeneely/swen331fuzzer/) fi XAMPP installation from the activity. You *must* make sure your code runs on the CI successfully! Please note that the version o should make sure your application works on these environments.
- Along with functionality, your applications and instructions should be clear and easy to understand and use. Imagine that you a proficient customer. Instructions should be clear, concise and unambiguous. Write a clear README.txt file that includes specif
- If need be, you may modify your GitLab CI file to do any custom installs. For example `MechanicalSoup` is not installed by defau `MechanicalSoup` to your `before_script` clause (note that it's `pip3` not `pip`)

Part 0: –custom-auth and CLI

For this initial part, you will need to implement the –custom-auth feature to log into DVWA. Implicit in this is command line parsing command line parsing libraries like Ruby's optparse and Python's argparse, or parse yourself).

This fuzzer should work on any web application. But, when `--custom-auth=dvwa` is given, your fuzzer will know the location of the hardcoded). With custom authentication turned off, the fuzzer should just crawl the exterior of the webapp (perhaps get lucky if the

For the DVWA custom auth, your fuzzer will need to do the following sequence of operations automatically:

1. Go to `{URL}/setup.php` where `{URL}` is the given url from the command line that points to a DVWA instance
2. "Click" on the Create/Reset Database (i.e. submit the form)
3. Go to `{URL}`, and it will forward you to the login page
4. Enter in "admin" and "password"
5. "Click" Login (i.e. submit the form)
6. Go to the DVWA Security page (`{URL}/security.php`)
7. Select "Low" and submit the form
8. Begin your fuzzing operations. (See rest of instructions)

You *cannot* assume that DVWA will be installed on a specific server, in a specific folder, on a specific port. For example, `http://exa`

Be sure to document in your README any setup you need. If your code does not work on the CI, the grader will be running this loca

To demonstrate you are logged in and security is correctly set, go back to the home page and print out the contents of the HTML of submitting Part0.

For part 0, no need to handle the "required" options like `--common-words` just yet - these will be required in Part 1.

When you are done with your implementation (i.e. it is working, and ready to grade), push your code to gitlab (make sure it works ir continue pushing commits after this tag - the tag is just a mechanism that lets us know which commit should count as your submis

Part 1: fuzz discover

On the discovery side, your fuzzer will need to discover as many potential inputs to the system as possible. It will need to do the fol

- **Page discovery**. The fuzzer must crawl and guess pages.
  - **Page Guessing**. The fuzzer should use the common word list to discover potentially unlinked pages. Attempt every combi words and extensions are in text files referred to in your command line arguments. Your guessing may limit itself to the r page (although a real fuzzer should!). For example, if the given url was `http://localhost/foo` , and your word list has `ba` `http://localhost/foo/bar` and `http://localhost/foo/baz` (and you do NOT need to construct other permutations lik
  - **Link Crawling**. Starting from the initial URL given, and from any page guessed, the fuzzer should discover pages on the si URLs that your fuzzer confirms exist. Do not follow any links off-site in your crawl. Do not go into an infinite loop. Beware aware of what DVWA's logout page is and skip crawling it.
- **Input discovery**. On every page discovered, the fuzzer should attempt to discover every possible input into the system.
  - **Parse URLs**. The fuzzer should be able to take a URL and parse it down to manipulate its input and recognize which page `something=a` is the same page as `http://localhost/index.jsp?anotherthing=b` , and there are two inputs that can be f
  - **Form parameters**. All input fields to forms should be considered inputs.
  - **Cookies**. Cookies are values that the application write to the browser cache, then reads from later. Since the application r (e.g. DVWA uses a cookie to set High/Medium/Low security)

When you are done with your implementation (i.e. it is working, and ready to grade), push your code to gitlab (*make sure it works in*

Part 2: fuzz test

Once you're done with input discovery, it's time to test. Testing has two parts: trying vectors, and then determining if the outcome

To conduct your test, you must use an **external list of fuzz vectors**. These are strings of common exploits to vulnerabilities. These l (https://wiki.owasp.org/index.php/OWASP_Testing_Guide_Appendix_C:_Fuzz_Vectors) is a fine place to start. **TIP**: when develo you *know* are vulnerable, so unnecessary vectors can slow your Edit-Compile-Test cycle.

Upon sending in the vector, you'll need examine the response to see if the page may have a vulnerability. Here are some reasons, an

- **Lack of sanitization**. Given inputs with data that should be sanitized, the fuzzer should report whether or not the data was act delimited list is not specified by the `--sanitized-chars` command line option.
  - For example, your fuzzer could try a string like `foobar<foobar` and if the resulting page had sanitization, then that string string `foobar<foobar` is still in the resulting page (i.e. the dangerous character was not sanitized, and you know it's not a normally), then you you know the page had a lack of sanitization.
- **Sensitive data leaked**. As a tester, you might have some test data that you know is sensitive and should never be leaked. The sy request and response if that data has been disclosed. This is in an external text file and provided on the command line (`--sens` should never see like "MySQL" as well as personal information like a social security number. Add "MySQL" and "123-45-6789" (w feel you need for your testing.
- **Delayed response**. If the response takes longer than a pre-defined threshold, then there's a potential of a denial-of-service vul
- **HTTP response codes**. If the HTTP response code is not OK (i.e. 200), then something went wrong. Report it. This can happen HTTP codes to be human readable

When you are done with your implementation (i.e. it is working, and ready to grade), push your code to gitlab (*make sure it works in*

Grading

Given that we are fuzzing *known* vulnerable applications, we will run your fuzzer to ensure that it reports errors on those vulnerabi vulnerabilities in specific applications - the point of this exercise is to make your fuzzer *discover* potential vulnerabilities. The gradi

- (25 pts) –custom-auth=dvwa
- (45 pts) fuzz discover
- (30 pts) fuzz test
- Each deliverable must include simple and clear setup steps via README
- Each deliverable must include meaningful, easy to read output

Specifically, we will be assigning points to these categories:

Part 0 (25 pts)

- Shared Fuzzer in gitlab: 1
- On Time: 3
- Runs on CI: 2
- Correct DB Init: 2
- Custom Auth Implemented: 4
- Authenticate into DVWA: 4
- Readme: 3
- Print homepage: 3
- Clean code: 3

Part 1 (45 pts)

- On Time: 2
- Runs on CI: 3
- Link Discovery: 4
- No External Links: 2
- Page Guessing: 4
- Found Pages: 3
- Common Words list: 2
- Guess Pages Crawled: 2
- Parse URLs: 4
- Duplicate URLs handled correctly: 2
- Form Parameters: 4
- Cookies: 4
- Readme: 3
- Output: 3
- Clean code: 3

Part 2 (30 pts)

- On Time: 2
- Runs on CI: 2
- Tests with vectors: 5
- Tests with sanitized-chars: 4
- Tests with sensitive data leaks: 4
- Use parameter for slow response: 2
- Reports non-200 response codes: 3
- Readme: 2
- Clean/ readable output: 3
- Clean code: 3

Submission

Submit your source code via GitLab. We will pull from the `master` branch at the deadline (if you need us to pull from a different bra
README on getting the code to run in other environments.